UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Learned Image Compression

Marco Cagnazzo

Multimedia Communications

Outline

Outline

What is Learned Image Compression (LIC)?

Definition

Learned Image Compression (LIC), or *Neural Image Compression* (NIC) refers to the application of neural networks and machine learning to data compression tasks

Why "Learned"?

- Traditional codecs (JPEG, JP2K,. . .) use **hand-crafted** rules
- LIC algorithms are learned **end-to-end** from data using generative models (VAEs, GANs, etc.)

State of the Art

While neural methods were once experimental (late '80s), modern models now **outperform** the best handcrafted codecs (like JP2K) in international challenges

Evolution of the Paradigm

Classical Approach (JPEG / JPEG 2000)

Relies on **hand-crafted** linear transforms (DCT, Wavelets) designed by experts to de-correlate pixels based on general statistical assumptions

Neural Approach

Relies on **learned** non-linear transforms. The mapping from pixels to coefficients is optimized directly from data

- **Linearity:** Classical transforms are linear; NIC uses non-linear activations (ReLU, GDN) to capture complex structures
- **Adaptivity:** Classical standards adapt via parameters (e.g., custom Q-tables) but rely on fixed basis functions. LIC learns the transform directly from the data distribution

The Linear Limit: KLT vs. Neural

The KLT is the optimal linear transform for Gaussian data because it perfectly decorrelates components.

KLT (Linear Optimal)

- Data-dependent but **linear**
- Optimal only for Gaussian sources
- Computationally heavy for large blocks

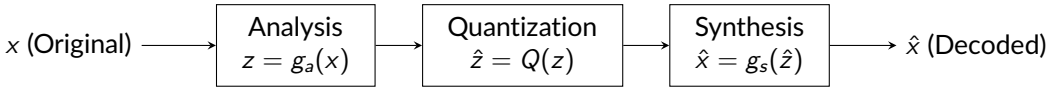
Neural (Non-linear)

- Learns a **non-linear manifold**, capturing high-order dependencies (edges, textures) that linear methods miss
- CNN architectures with weight-sharing process large receptive fields without block-based constraints.

Key Insight: NIC can be viewed as a *Non-linear KLT* optimized for Rate-Distortion

A Generalized Coding Architecture

Even with the transition to deep learning, the core structural flow of transform coding remains remarkably consistent:



- **Legacy Frameworks**: The analysis and synthesis steps rely on fixed linear operations like DCT or DWT.
- **Learned Paradigms**: These stages are replaced by non-linear neural networks (g_a and g_s) that are optimized directly from data.

Two Philosophies of R-D Optimization

The transition to learned compression marks a shift in how we approach the Rate-Distortion optimization:

Combinatorial Selection (Classical)

Optimization is a **search process** over a discrete set of options.

- We choose the best configuration from **predefined** tools (e.g., selecting quantization steps in JPEG or bit-plane truncation in EZW/EBCOT).
- The "dictionary" of transforms and filters is fixed; we only optimize their usage.

Continuous Learning (Neural)

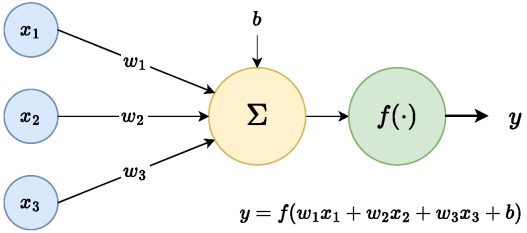
Optimization is a **design process** over a continuous space.

- We use gradient descent to physically **shape the filters** (weights) of the network by minimizing a differentiable loss function: $\mathcal{L} = D(x, \hat{x}) + \lambda R(\hat{z})$
- The transform itself is "born" from the data and is tailored to the specific bit-rate target (λ).

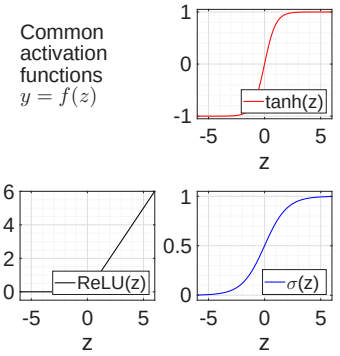
Outline

Fundamentals: The Artificial Neuron

The foundational building block of neural networks is the artificial neuron. It computes a weighted sum of incoming signals, adds a bias term, and applies a non-linear activation function $f(\cdot)$ to produce the output: $y = f(\mathrm{w}^T\mathrm{x} + b)$.



Common activation functions $y = f(z)$



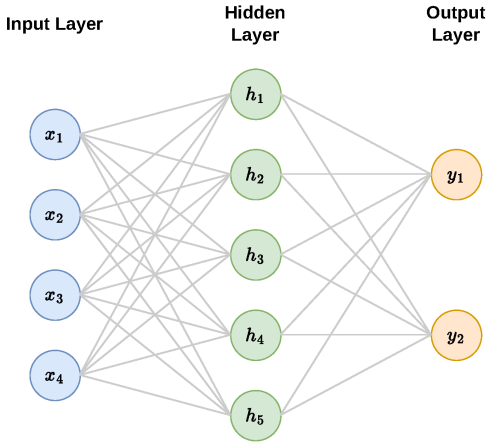
The Multilayer Perceptron (MLP)

In an MLP, layers are **Fully Connected**: every neuron connects to all neurons in the previous layer. Computations are heavily vectorized using weight matrices (W) and bias vectors (b). For the network shown here:

$$h = f(W_1 x + b_1)$$

$$y = g(W_2 h + b_2)$$

This sequential matrix multiplication is exactly what enables efficient GPU acceleration.



Gradient descent and backpropagation

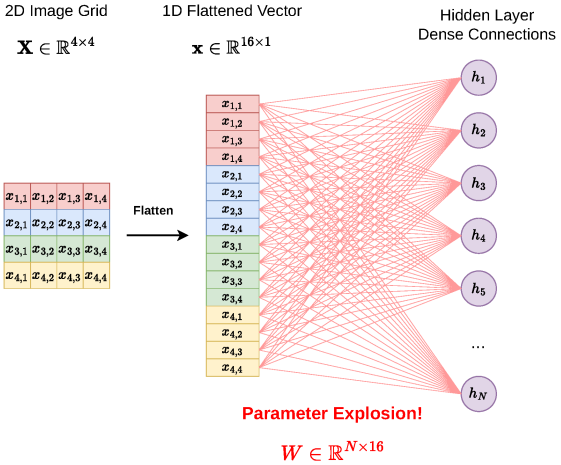
- Learning is an optimization problem. We define a Loss Function $\mathcal{L}(\theta)$ quantifying the prediction error and update the parameters θ (weights and biases) iteratively against the gradient:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

- In deep architectures, finding which hidden weights caused an error is challenging. **Backpropagation** solves this by systematically applying the calculus chain rule from the output backwards, providing the exact gradient for every parameter.

NN and images

Flattening a 2D image into a 1D vector treats all pixels as independent inputs, ignoring their spatial topology.
Near and far pixels are treated in the same way



The Dimensionality Curse of MLPs

The cost of ignoring Spatial Context:

- **No Translation Invariance:** Without a specific structure, the model is forced to redundantly learn to recognize the same pattern independently at every possible location.
- **Parameter Explosion:** Connecting every pixel to every neuron in the next layer leads to millions of weights even for small images.
- **Training Difficulty:** This over-parameterization makes the model highly prone to overfitting and computationally intractable to train.

Challenge: We need an architecture that "understands" proximity and can reuse learned patterns.

Convolutional Neural Networks (CNNs)

Why CNNs?

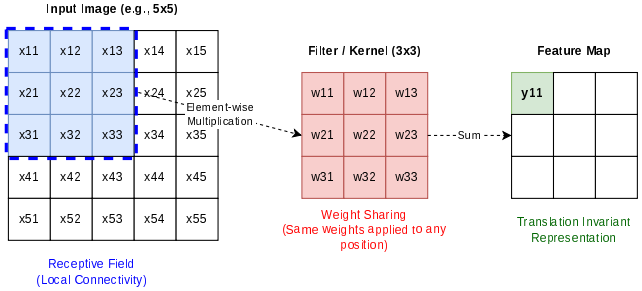
CNNs solve the scalability wall by embedding a **spatial prior** directly into their architecture, naturally exploiting local correlations.

Key Architectural Features:

- **Local Connectivity & Weight Sharing:** Instead of dense connections, small filters slide across the image. This guarantees **translation invariance** and drastically reduces the parameter count.
- **Strided Convolutions (Analysis):** Using a stride $s > 1$ reduces spatial dimension , compacting information into multiple channels (learned downsampling).
- **Transposed Convolutions (Synthesis):** Learned upsampling operations that reconstruct the full-resolution image from the compressed latent tensor.

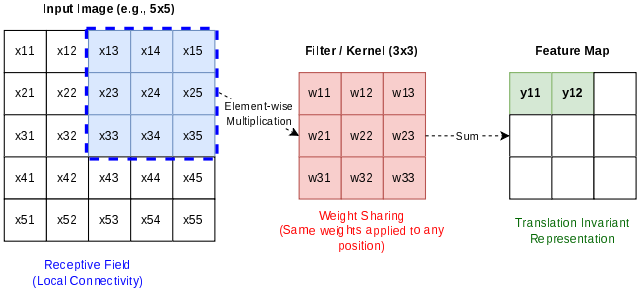
Visualizing the Convolution

A small filter (kernel) slides across the input image. At each step, it performs an element-wise multiplication and sum to produce a single value in the output feature map



Visualizing the Convolution

A small filter (kernel) slides across the input image. At each step, it performs an element-wise multiplication and sum to produce a single value in the output feature map We can “skip” some pixels (stride) compacting the visual information



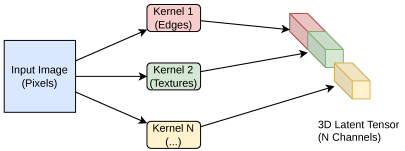
Parallel Feature Extraction

The Logic of "How Many?"

A single kernel acts as a specific detector (e.g., a horizontal edge). However, natural images are too complex to be described by just one type of feature.

Building the 3D Latent Tensor:

- **Parallel Kernels:** We apply **multiple independent kernels** to the same input area simultaneously.
- **Channels as Features:** Each output "channel" (or feature map) represents the activation of one specific detector across the image.
- **Dimensionality:** This transforms the 2D input into a **3D Tensor** (Height $\times$ Width $\times$ Channels).



Generalized Divisive Normalization (GDN)

Instead of simple activations (like ReLU), neural codecs use **GDN**
For a feature v_i, the normalized output w_i is: $w_i = \frac{v_i}{\sqrt{\beta_i + \sum_j \gamma_{ij} v_j^2}}$

Where β_i and γ_{ij} are learned parameters, and the sum is over neighboring features/channels.

Why does it work better for compression?

- **Lateral Inhibition:** The denominator measures **local energy**. If neighboring features (v_j) are large, the response of v_i is suppressed.
- **Link to Perception:** This mathematically models **masking effects** in the human visual system (a strong stimulus hides weaker nearby stimuli).
- **Gaussianization:** It decorrelates the signal, making the latent distributions roughly Gaussian, which is optimal for **entropy coding**.

Transposed Convolutions

To reconstruct the image from the compressed latents, the Decoder needs to increase the spatial resolution. This is achieved via **Transposed Convolutions Learned Upsampling**:

Unlike fixed interpolation (e.g., bilinear), transposed convolutions **learn** the optimal basis functions to reconstruct the signal from the latent space.

The Mechanism (Broadcasting):

- **1-to-N Mapping:** A single value from the input latent tensor is multiplied by the kernel weights and "stamped" onto the output grid.
- **Fractional Stride:** Using a stride $s > 1$ dictates how far apart these kernels are stamped on the output, effectively **increasing the spatial dimensions**.
- **Overlap & Sum:** Where the stamped kernels overlap in the output grid, their values are summed together to form the final reconstructed pixels.

Outline

The Core Idea: From Pixels to a Latent Tensor

Instead of transforming fixed blocks (like 8×8 in JPEG), neural compression transforms the entire image into a **deep 3D Latent Tensor**.

How the Tensor is built:

- **Spatial Reduction:** Strided convolutions compress the spatial dimensions (W, H).
- **Channel Expansion:** To avoid losing information, the network learns multiple **parallel features** (Channels). Each channel represents a different structural aspect of the image.

The JPEG AI Example

To achieve state-of-the-art compression, **JPEG AI** maps the input image into a latent tensor with **160 channels** for Luminance and **96 channels** for Chrominance.

The Autoencoder Framework

In learned compression, the traditional transform coding pipeline is replaced by an **Autoencoder** architecture:

- **Analysis Transform (g_a):** The Encoder. It uses CNNs to map the image x to the latent tensor y.
- **Bottleneck:** This is where the compression happens. The latents y are quantized to $\hat{y}$ to be entropy-coded.
- **Synthesis Transform (g_s):** The Decoder. It learns to reconstruct the image $\hat{x}$ from the quantized latents $\hat{y}$.

Terminology Bridge

What we called "Transform Coefficients" in JPEG are called **"Latent Variables"** in Neural Compression.

The Rate-Distortion VAE

Modern neural codecs are instances of **Variational Autoencoders (VAEs)** optimized for a specific objective.

The VAE-Compression Connection:
Compression aims to minimize the **Lagrangian**: $\mathcal{L} = R + \lambda D$.

Optimization Objective

$$\mathcal{L} = \underbrace{D_{KL}[q(\hat{y}|x)||p(\hat{y})]}_{\text{Rate (R)}} + \lambda \underbrace{\mathbb{E}[\rho(x, \hat{x})]}_{\text{Distortion (D)}}$$

λ controls the trade-off: high $\lambda \rightarrow$ high quality, low $\lambda \rightarrow$ high compression.

Why Compress in the Latent Space?

Instead of quantizing pixels directly, we quantize the output of the Analysis Transform ($y = g_a(x)$).

In Pixel Space:

- High redundancy.
- Difficult to model probabilities.
- Quantization errors are highly visible (blocking).

In Latent Space:

- Features are **decorrelated**.
- Distributions are more predictable (Gaussian/Laplace).
- The network can hide quantization noise in less "perceptually important" channels.

The Non-Differentiability Problem

The quantization operator $\hat{z} = Q(z)$ is a stair-case function.

The Gradient Issue

- The derivative is **zero** almost everywhere
- Backpropagation cannot pass through the quantizer to update the encoder weights

The Mismatch: We need "hard" quantization at test time (to get bits), but we need "soft" or differentiable operations during training

Solution: Additive Uniform Noise

To enable training, we replace rounding with Additive Uniform Noise

The Proxy Operation

During training: $\hat{z} \approx z + u$, where $u \sim \mathcal{U}(-0.5, 0.5)$

Why does this work?

- Adding noise makes the operation **differentiable**
- It acts as a continuous relaxation of the discrete quantization proces
- **Mathematical Link:** Quantization noise acts like independent white noise at high resolution

The Scale Hyperprior: Spatial Adaptability

The Limit of the Basic VAE: A fixed probability model assumes all latents are independent. In reality, latents exhibit strong spatial correlation in their **variance** (scale).

The Hyperprior Concept

We transmit additional side-information $\hat{z}$ (the **Hyperprior**) to dynamically predict the probability distribution of the latents $\hat{y}$ at each spatial location.

The Mechanism (A Second Autoencoder):

- **Hyper-Encoder (h_a):** Operates in parallel to the main encoder (g_a). It analyzes the latents y to extract a compressed hyper-latent z.
- **Hyper-Decoder (h_s):** Decodes $\hat{z}$ to predict the **standard deviation** (σ) for the conditional prior: $p(\hat{y}|\hat{z}) \sim \mathcal{N}(\mu, \sigma^2)$

Outline

JPEG AI: Standardizing Neural Compression

Standardized as **ISO/IEC 29170-2**, JPEG AI is the first international standard for image coding based entirely on **Deep Learning** technologies.

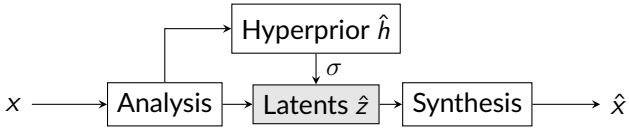
The Paradigm Shift

While legacy standards (JPEG, JPEG 2000) are built on hand-crafted linear transforms, JPEG AI is the industrial crystallization of the **Nonlinear Transform Coding** paradigm.

- **Timeline:** Developed between 2020 and 2024, finalized in early 2025.
- **Objective:** To outperform HEVC (Intra) and VVC (Intra) by 50% in coding efficiency while supporting both human and machine tasks.

Methodology: The Hierarchical VAE

The technical backbone of JPEG AI is a refined version of the **Scale Hyperprior Autoencoder**.



- **Analysis/Synthesis Transforms:** Deep CNNs optimized for Rate-Distortion.
- **Quantization:** Uses rounding proxies during training to handle gradients.
- **Side Information:** The hyperprior ($\hat{h}$) captures the non-stationary statistics of the image, acting as a dynamic "entropy-coding guide".

Multimodality: Human vs. Machine Vision

One of the most innovative features of JPEG AI is its **dual-use bitstream**.

The "AI-Ready" Design

A single compressed file can be decoded in two different ways:

1. **Human Vision Decoder:** Reconstructs standard pixels for visual consumption (optimized for high MS-SSIM).
2. **Computer Vision Task Decoder:** Extracts features directly from the latent space for tasks like **Object Detection** or **Segmentation** without full pixel reconstruction.

Benefit: Massive reduction in computational overhead for large-scale AI pipelines (e.g., smart cities, autonomous vehicles).

Performance: The Rate-Distortion Leap

Experimental validation by the JPEG Committee shows that **JPEG AI** fundamentally shifts the R-D curve compared to classical codecs.

Coding Gain:Â

- Up to **27% bit-rate saving** compared to VVC/H.266 (the current state-of-the-art video codec).
- Over an **order of magnitude** savings compared to the original JPEG (1992) for the same perceived utility.

Visual Quality at Low Rates

At typical coding rates ($\sim$**0.5 bpp**), JPEG AI perfectly preserves high-frequency textures that are often smoothed or "washed out" by traditional methods

Performance: Hardware Scalability

Beyond bit-rate, a standard must be deployable. JPEG AI introduces a novel approach to computational complexity.

Complexity Profiles (kMAC/pixel):

- Complexity is measured in **kilo-Multiplication Accumulator Count** per pixel, a standard metric for neural network deployment.
- The standard defines multiple synthesis transforms that decode the **same bitstream** at different quality/complexity tiers.

- **Dec0 (8 kMAC/px):** Optimized for low-end CPUs.
- **Dec1 (23 kMAC/px):** Designed for real-time decoding on mid-range smartphones.
- **Dec2 (214 kMAC/px):** Transformer-based, targets high-end GPUs for maximum visual fidelity.

Outline

Conclusions: The Paradigm Shift

Learned Image Compression (LIC) represents a fundamental methodological departure from classical codec design.

- **End-to-End Optimization:** Instead of manually designing and patching disparate blocks (transforms, quantization, entropy), the entire pipeline is jointly optimized for the Lagrangian $\mathcal{L} = R + \lambda D$.
- **Data-Driven Transforms:** The architecture learns non-linear, content-adaptive basis functions rather than relying on fixed linear models (like the DCT).

State of the Art: The Grand Trade-off

Pros: Unprecedented Rate-Distortion efficiency, officially surpassing the best classical video codecs (VVC) in static image coding.
Cons: Massive computational weight. It requires millions of MAC operations per pixel, strictly demanding parallel hardware accelerators (GPUs/NPUs).

Current Challenges and Limitations

Despite the R-D superiority, neural codecs face critical engineering hurdles before ubiquitous adoption.

Key Bottlenecks:

- **Energy Consumption:** The high kMAC/pixel ratio translates directly to battery drain on mobile devices, making high-resolution video deployment currently prohibitive.
- **Cross-Platform Determinism:** Classical codecs guarantee bit-exact reconstruction. Neural networks often rely on floating-point math, which can yield varying rounding results across different GPUs, leading to severe **decoder drift**.
- **Out-of-Distribution Vulnerability:** Since transforms are learned from datasets, highly unusual signals (e.g., specific scientific imagery, severe sensor noise) might be poorly modeled or reconstructed with visual "hallucinations".

Future Horizons in Neural Compression

The field is rapidly expanding beyond simple mathematical Rate-Distortion metrics.

Emerging Research Vectors:

- **Generative Compression:** Integrating GANs and Diffusion Models to synthesize perceptually perfect textures at ultra-low bitrates, prioritizing *realism* over exact pixel-by-pixel fidelity.
- **Coding for Machines (Task-Oriented):** Compressing data not for human eyes, but for downstream AI tasks. Latent tensors can be fed directly to classifiers or autonomous driving networks without ever reconstructing the RGB image.
- **Hardware-Algorithm Co-design:** As NPUs become ubiquitous in mobile SoCs, future codecs will be co-designed directly with silicon engineers to execute non-linear transforms with near-zero overhead.